Celebal Summer Internship 2026

Week 2 Task: E-Commerce Sales Database

ASSIGNMENT – 2

SECTION – A

SQL BASICS (SELECT, CONSTRAINTS, PRIMARY KEYS)

Q1. Write a query to display all columns and rows from the customer's table.

ANS:- SQL QUERY WITH OUTPUT

```
Find
1 •  SELECT * FROM customers;
2
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

customer_id	first_name	last_name	email	city	state	join_date	is_premium
101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0
103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1
104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0
105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1
106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0
107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	0
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

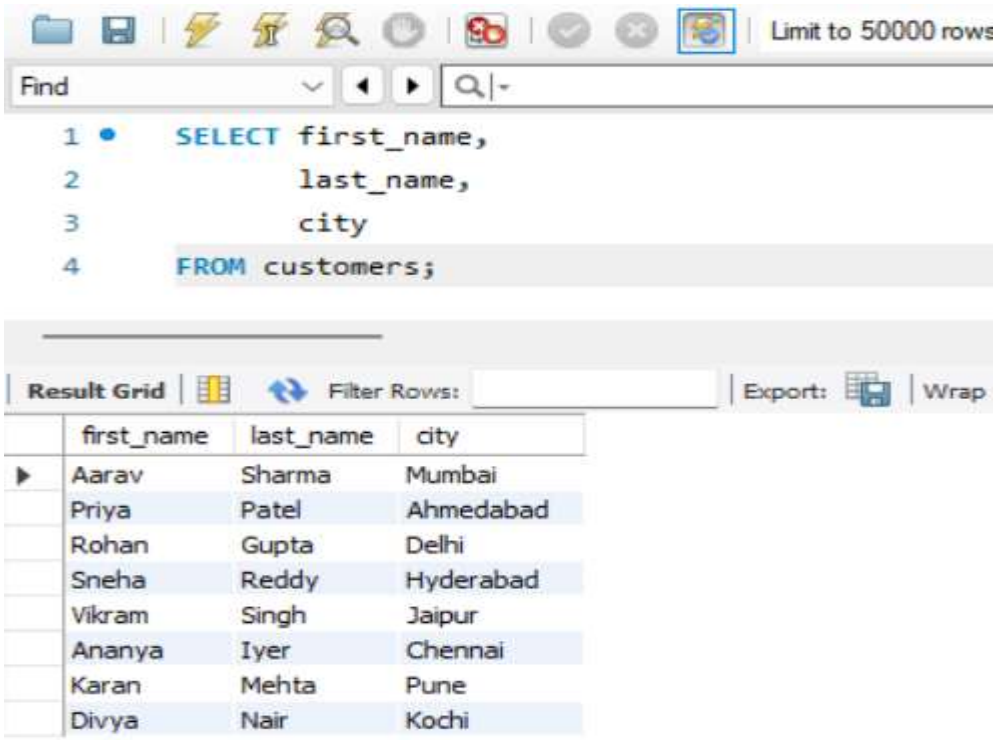
Explanation

- SELECT is used to retrieve data from a table.
- * means all columns.
- FROM customers specifies the table from which data will be retrieved.

- This query displays every row and every column in the customers table.

Q2. Retrieve only the first_name, last_name, and city of all customers.

ANS:- SQL QUERY WITH OUTPUT



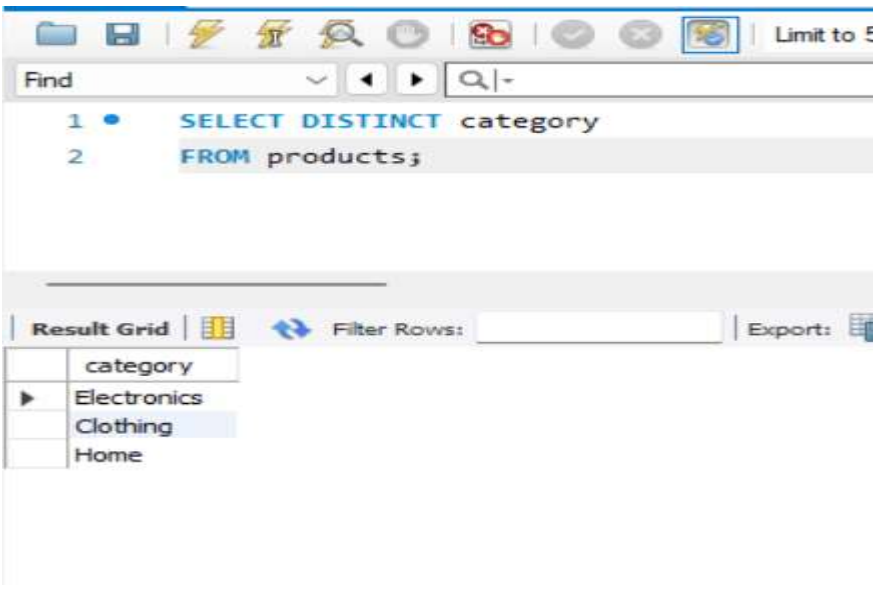
first_name	last_name	city
Aarav	Sharma	Mumbai
Priya	Patel	Ahmedabad
Rohan	Gupta	Delhi
Sneha	Reddy	Hyderabad
Vikram	Singh	Jaipur
Ananya	Iyer	Chennai
Karan	Mehta	Pune
Divya	Nair	Kochi

Explanation

- Instead of selecting all columns, only specific columns are selected.
- This improves readability and performance when only required information is needed.
- Commonly used in reports and dashboards.

Q3. List all unique categories available in the products table.

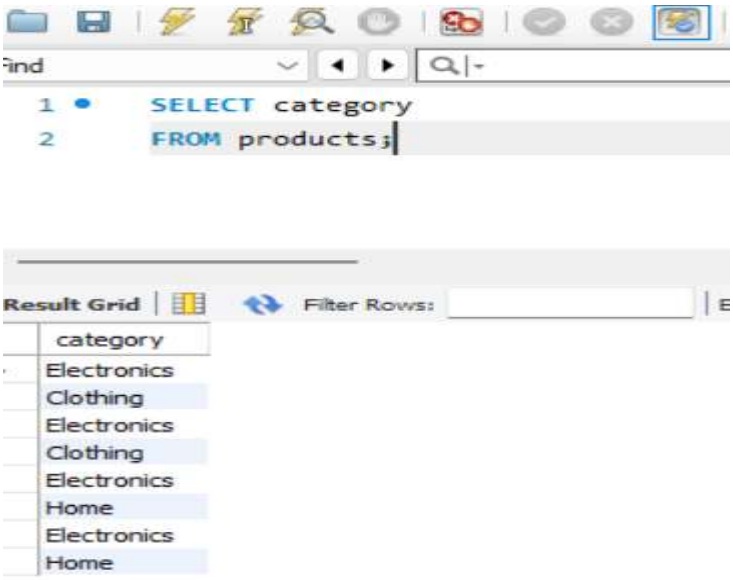
ANS:- SQL QUERY WITH OUTPUT (With DISTINCT)



Explanation

- DISTINCT removes duplicate values.
- The products table contains multiple products in the same category.
- Without DISTINCT, categories would appear repeatedly.
- This query returns only unique category names.

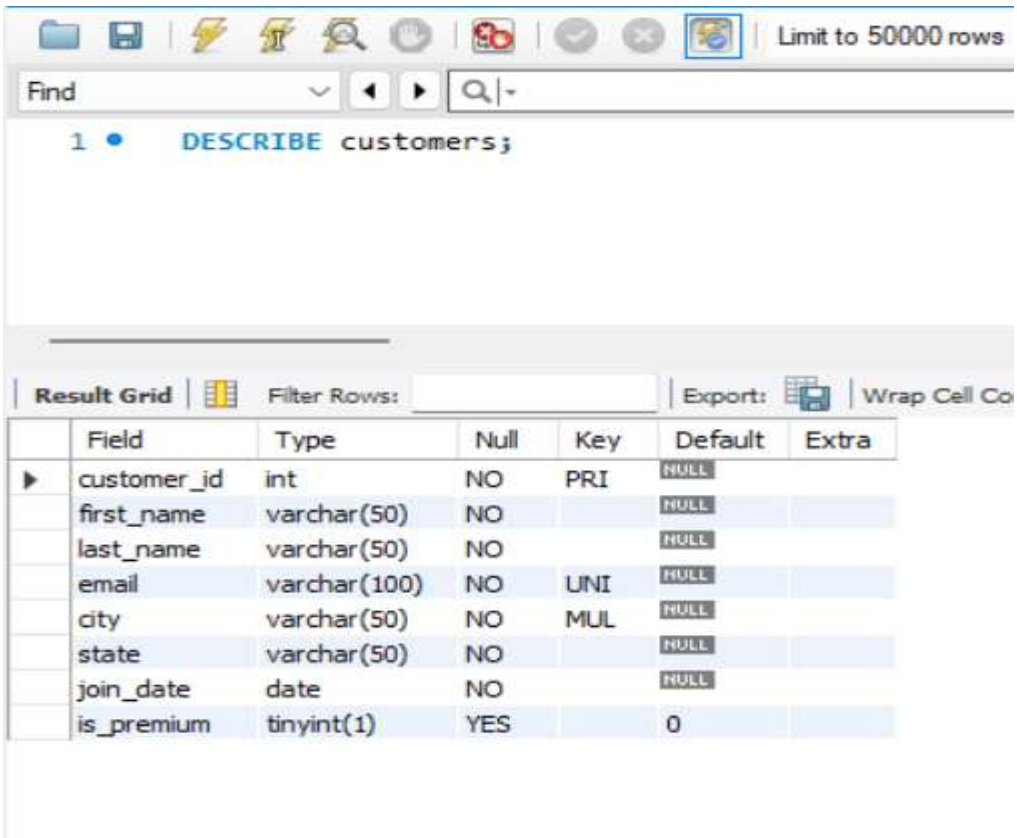
(WITHOUT DISTINCT)



Q4. Identify the Primary Key of each table in the schema. Explain why a Primary Key must be unique and NOT NULL.

ANS:- **Primary Keys**

Primary Key	Table Name
customer_id	customers
product_id	products
order_id	orders
item_id	order_items



Field	Type	Null	Key	Default	Extra
customer_id	int	NO	PRI	NULL	
first_name	varchar(50)	NO		NULL	
last_name	varchar(50)	NO		NULL	
email	varchar(100)	NO	UNI	NULL	
city	varchar(50)	NO	MUL	NULL	
state	varchar(50)	NO		NULL	
join_date	date	NO		NULL	
is_premium	tinyint(1)	YES		0	

Explanation

A Primary Key uniquely identifies every record in a table.

Why must a Primary Key be UNIQUE?

If duplicate values exist, the database cannot distinguish between records.

Example:

customer_id	first_name
101	Aarav
101	Priya

The database would not know which record is correct.

Why must a Primary Key be NOT NULL?

Every record must have an identifier.

Example:

customer_id	first_name
NULL	Aarav

A NULL value cannot uniquely identify a row.

Therefore, Primary Keys must always be UNIQUE and NOT NULL.

Q5. What constraints are applied to the email column? What happens if a duplicate email is inserted?

ANS:- Constraint Definition

```
email VARCHAR(100) UNIQUE NOT NULL
```

Constraints Applied

1. **UNIQUE Constraint**
    - Prevents duplicate email addresses.

2. **NOT NULL Constraint**
    - Prevents empty email values.

SQL Query (Duplicate Email Test)

OUTPUT:-

ERROR 1062 :-

Duplicate entry 'aarav.s@email.com'

for key 'email'

Explanation

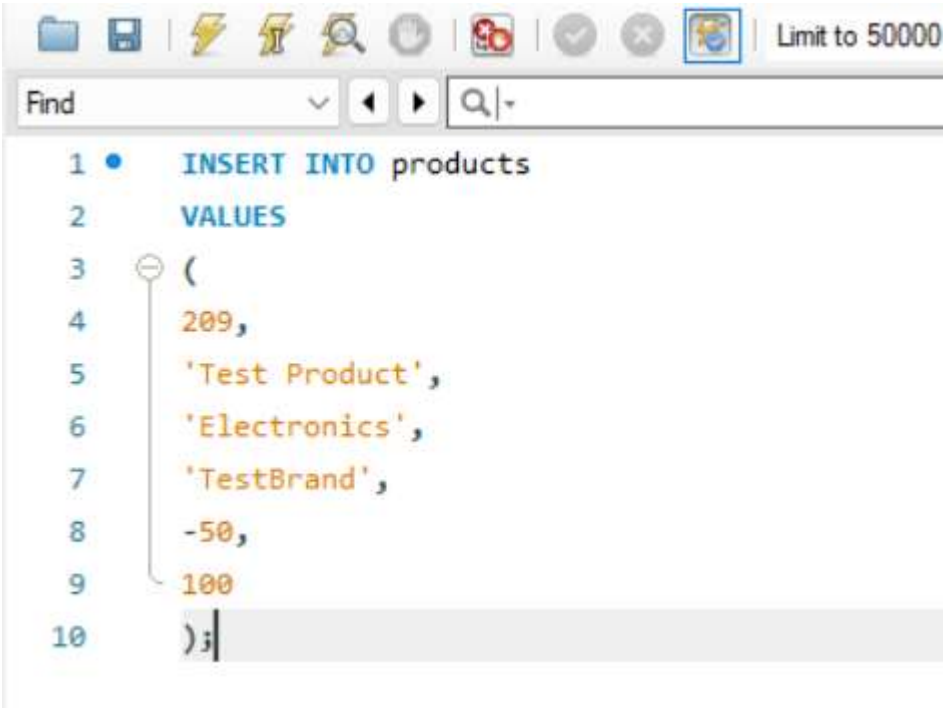
The email address aarav.s@email.com already exists in the customers table.

Because the email column has a UNIQUE constraint, MySQL rejects the insertion and displays an error.

This ensures that each customer has a distinct email address.

Q6. Insert a product with unit_price = -50. What happens and which constraint prevents it?

ANS:- SQL QUERY



Constraint Definition

OUTPUT

Explanation

The products table contains a CHECK constraint:

CHECK (unit_price > 0)

```
Find
1  CHECK (unit_price > 0)
```

This rule allows only positive values for product prices.

Since -50 is less than zero, the condition fails and MySQL prevents the record from being inserted.

Valid Example

```
Find
1   INSERT INTO products
2   VALUES
3   (
4   209,
5   'Test Product',
6   'Electronics',
7   'TestBrand',
8   500,
9   100
10  );
```

OUTPUT:-

```
85  13:09:05  INSERT INTO products VALUES ( 209, 'Test Product', 'Electronics', 'TestBrand', 500, 100 )   1 row(s) affected
```

Business Importance

The CHECK constraint ensures:

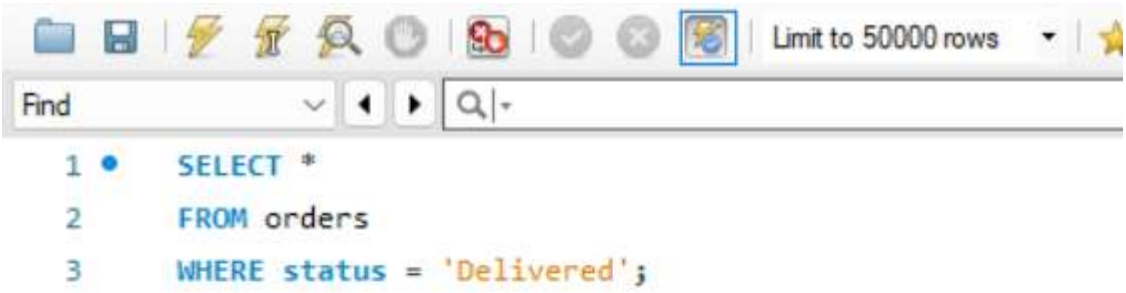
- Product prices remain valid.
- Incorrect data is prevented.
- Reports and sales analysis remain accurate.
- Data integrity is maintained.

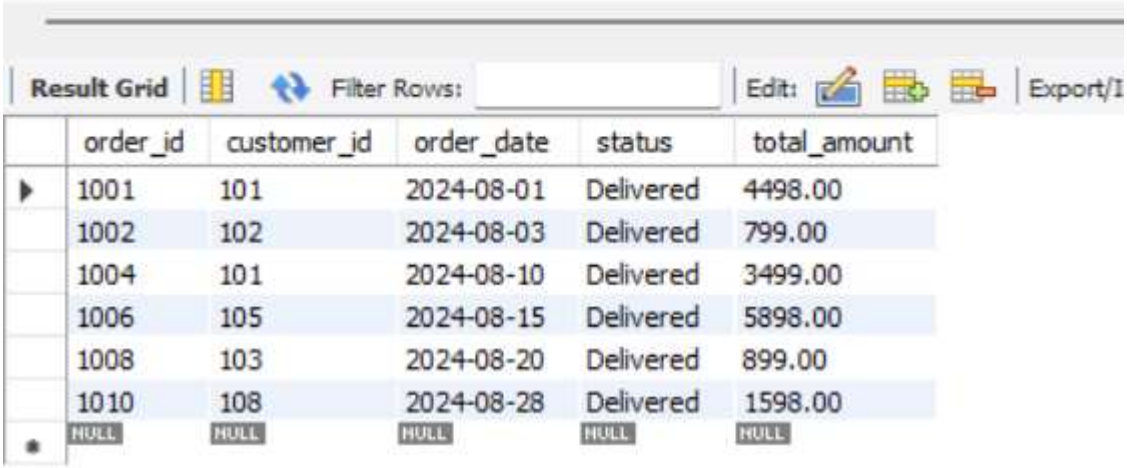
SECTION – B

FILTERING & OPTIMIZATION (WHERE, INDEXES)

Q7. Retrieve all orders with status = 'Delivered'

ANS:- SQL QUERY WITH OUTPUT





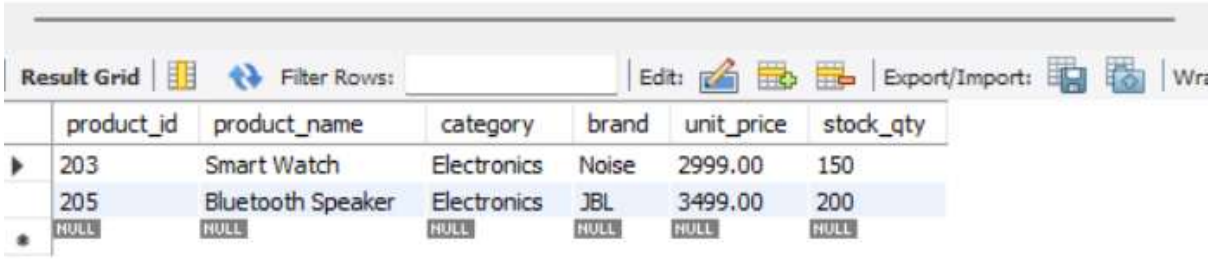
order_id	customer_id	order_date	status	total_amount
1001	101	2024-08-01	Delivered	4498.00
1002	102	2024-08-03	Delivered	799.00
1004	101	2024-08-10	Delivered	3499.00
1006	105	2024-08-15	Delivered	5898.00
1008	103	2024-08-20	Delivered	899.00
1010	108	2024-08-28	Delivered	1598.00
NULL	NULL	NULL	NULL	NULL

Explanation

- The WHERE clause filters rows based on a condition.
- Only orders whose status is **Delivered** are displayed.
- This query helps management analyze completed sales.

Q8. Find all products in the 'Electronics' category with a unit_price greater than ₹2000

ANS:- SQL QUERY WITH OUTPUT



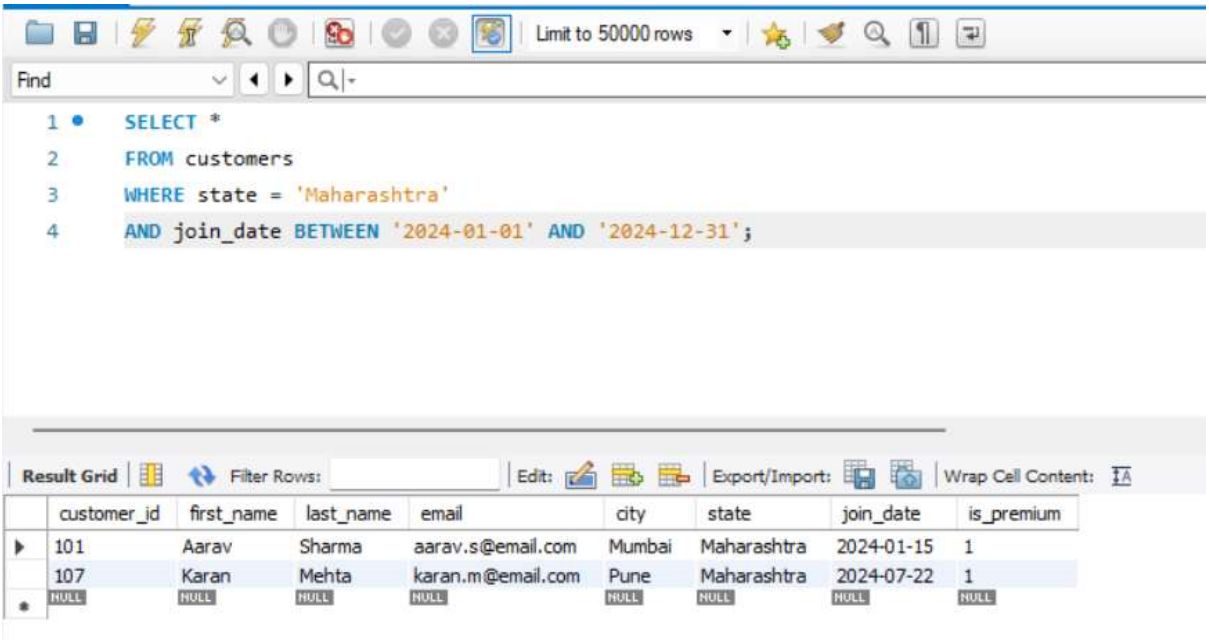
product_id	product_name	category	brand	unit_price	stock_qty
203	Smart Watch	Electronics	Noise	2999.00	150
205	Bluetooth Speaker	Electronics	JBL	3499.00	200
NULL	NULL	NULL	NULL	NULL	NULL

Explanation

- category = 'Electronics' selects only electronics products.
- unit_price > 2000 filters products costing more than ₹2000.
- AND ensures both conditions are satisfied.

Q9. List all customers who joined in the year 2024 and belong to the state 'Maharashtra'

ANS:- SQL QUERY WITH OUTPUT

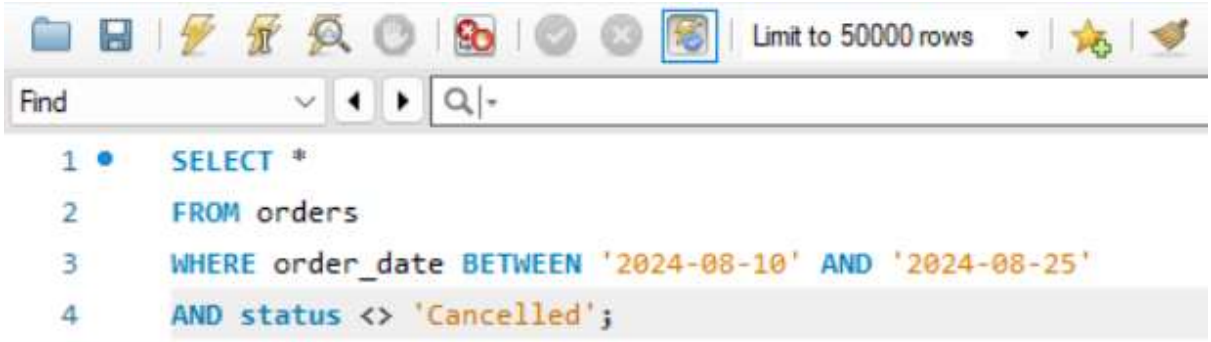


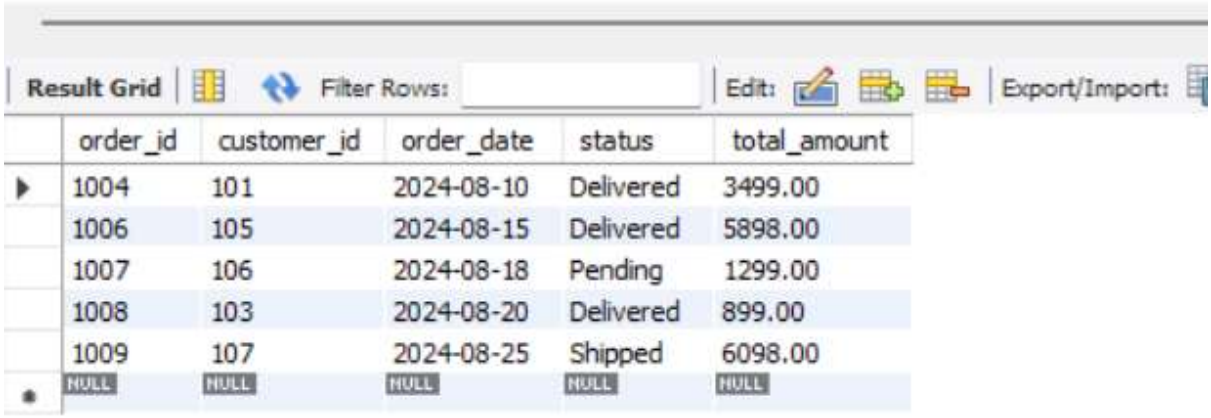
Explanation

- state = 'Maharashtra' filters customers from Maharashtra.
- BETWEEN selects customers who joined during 2024.
- This query combines multiple filtering conditions.

Q10. Find all orders placed between '2024-08-10' and '2024-08-25' (inclusive) that are NOT cancelled

ANS:- SQL QUERY WITH OUTPUT





order_id	customer_id	order_date	status	total_amount
1004	101	2024-08-10	Delivered	3499.00
1006	105	2024-08-15	Delivered	5898.00
1007	106	2024-08-18	Pending	1299.00
1008	103	2024-08-20	Delivered	899.00
1009	107	2024-08-25	Shipped	6098.00
NULL	NULL	NULL	NULL	NULL

Explanation

- BETWEEN includes both start and end dates.
- status <> 'Cancelled' excludes cancelled orders.
- Useful for analyzing active order activity during a specific period.

Q11. Explain what the index idx_orders_date does. How would it improve the performance of a query that filters orders by order_date? Write a sample query that would benefit from this index.

ANS:- INDEX CREATION

Explanation

The idx_orders_date index is created on the order_date column of the orders table.

An index works similarly to an index in a book. Instead of scanning every row in the table to find matching dates, MySQL can use the index to quickly locate the required records.

How It Improves Performance

Without the index:

- MySQL performs a **Full Table Scan**.
- Every row in the orders table is checked.
- Query execution becomes slower as the table grows.

With the index:

- MySQL directly searches the indexed dates.
- Fewer rows need to be examined.
- Query execution becomes much faster, especially for large datasets.

Sample Query Benefiting from the Index

```
SELECT *
FROM orders
WHERE order_date = '2024-08-15';
```

Result Grid | Filter Rows: | Edit:

order_id	customer_id	order_date	status	total_amount
1006	105	2024-08-15	Delivered	5898.00
NULL	NULL	NULL	NULL	NULL

Another Example Using a Date Range

```
SELECT *
FROM orders
WHERE order_date BETWEEN '2024-08-10' AND '2024-08-25';
```

Result Grid | Filter Rows: | Edit: | Export/Import:

order_id	customer_id	order_date	status	total_amount
1004	101	2024-08-10	Delivered	3499.00
1005	104	2024-08-12	Cancelled	2999.00
1006	105	2024-08-15	Delivered	5898.00
1007	106	2024-08-18	Pending	1299.00
1008	103	2024-08-20	Delivered	899.00
1009	107	2024-08-25	Shipped	6098.00
NULL	NULL	NULL	NULL	NULL

Conclusion

The idx_orders_date index improves the performance of queries that filter, search, or sort data using the order_date column. Instead of scanning the entire table, MySQL can quickly locate matching records through the index, resulting in faster query execution and better database performance.

Q12. If you run: SELECT * FROM customers WHERE YEAR(join_date) = 2024; — would the index on join_date be used? Explain why or why not, and rewrite the query to be index-friendly (SARGable).

ANS:- No, the index on the join_date column would generally **not be used efficiently** when the query uses YEAR(join_date) = 2024.

This is because the YEAR() function is applied directly to the indexed column. MySQL must calculate the year value for every row before comparing it with 2024. As a result, the database cannot effectively use the index and may perform a **full table scan**, which is slower for large tables.

Non-SARGable Query

```sql
SELECT *
FROM customers
WHERE YEAR(join_date) = 2024;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content

customer_id	first_name	last_name	email	city	state	join_date	is_premiu
101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0
103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1
104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0
105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1
106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0
107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	0
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Index-Friendly (SARGable) Query

Execute the selected portion of the script or everything, if there is no selection

```sql
SELECT *
FROM customers
WHERE join_date >= '2024-01-01'
AND join_date < '2025-01-01';
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

customer_id	first_name	last_name	email	city	state	join_date	is_premium
101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0
103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1
104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0
105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1
106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0
107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	0
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Explanation

The rewritten query directly compares the join_date values without applying any function. This allows MySQL to use the index efficiently, making the query faster and more scalable. Such queries are called **SARGable (Search Argument Able)** because they enable the database engine to utilize indexes for better performance.

Conclusion

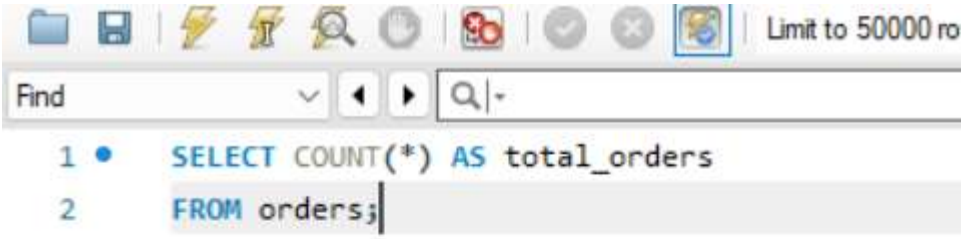
Using YEAR(join_date) = 2024 prevents efficient index usage, whereas using a date range condition allows MySQL to take advantage of the index on join_date, resulting in faster query execution and improved database performance.

SECTION – C

AGGREGATION (GROUP BY, SUM, COUNT, AVG, MIN, MAX)

Q13. Count the total number of orders in the orders table.

ANS:- SQL QUERY WITH OUTPUT



Explanation

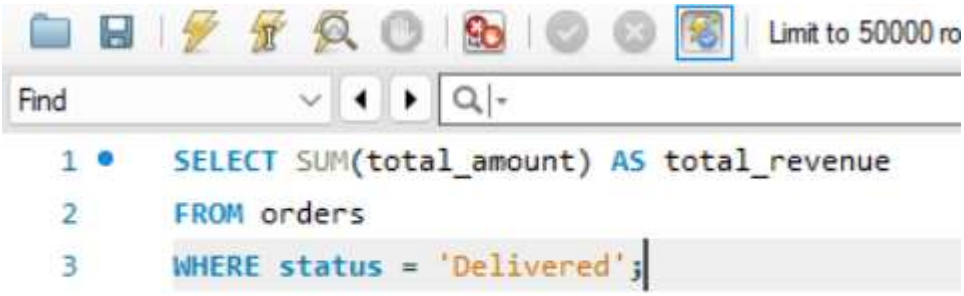
The COUNT(*) function counts the total number of rows in the orders table. Since there are 10 order records, the result is 10.

Conclusion

This query helps determine the total number of orders placed by customers

Q14. Find the total revenue (SUM of total_amount) from all 'Delivered' orders.

ANS:- SQL QUERY WITH OUTPUT



Calculation

4498 + 799 + 3499 + 5898 + 899 + 1598
= 17191

Explanation

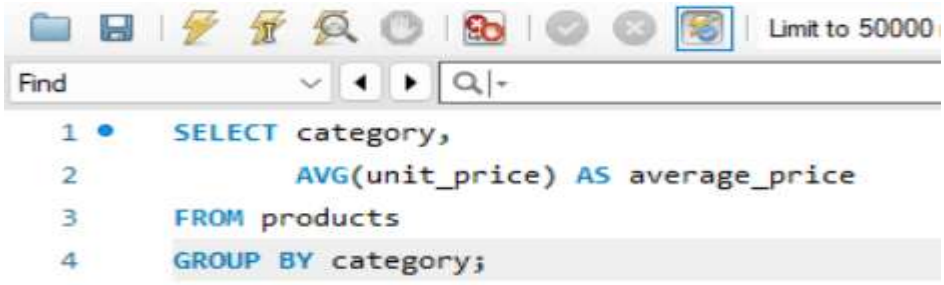
The SUM() function adds the values in the total_amount column for all orders whose status is 'Delivered'.

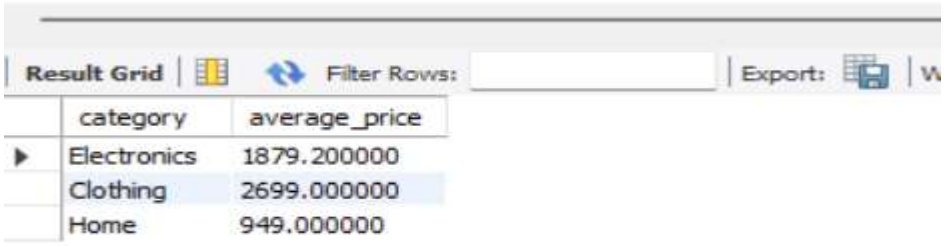
Conclusion

The total revenue generated from delivered orders is ₹17,191.

Q15. Calculate the average unit_price of products in each category.

ANS:- SQL QUERY WITH OUTPUT





Calculation

Electronics

1499 + 2999 + 3499 + 899 = 8896
8896 ÷ 4 = 2224

Clothing

799 + 4599 = 5398
5398 ÷ 2 = 2699

Home

1299 + 599 = 1898
1898 ÷ 2 = 949

Explanation

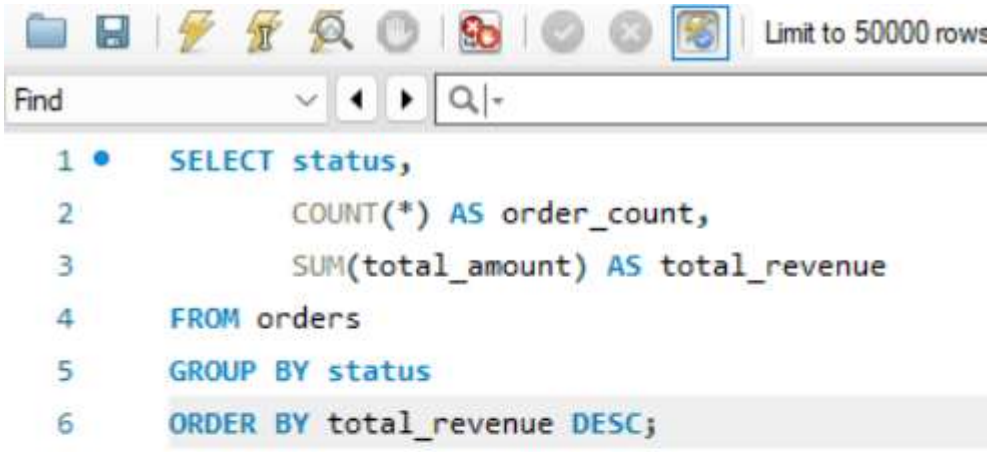
- AVG() calculates the average value.
- GROUP BY category creates separate groups for each category.

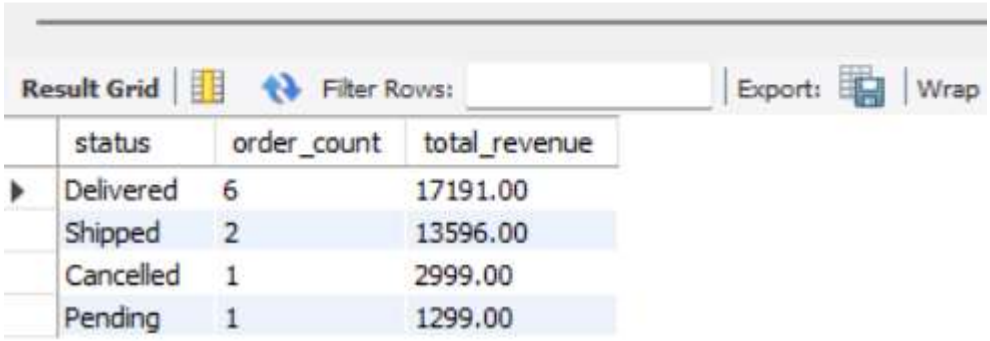
Conclusion

This query shows the average product price within each category.

Q16. For each order status, find the count of orders and the total revenue. Sort the result by total revenue in descending order.

ANS:- SQL QUERY WITH OUTPUT





status	order_count	total_revenue
Delivered	6	17191.00
Shipped	2	13596.00
Cancelled	1	2999.00
Pending	1	1299.00

Explanation

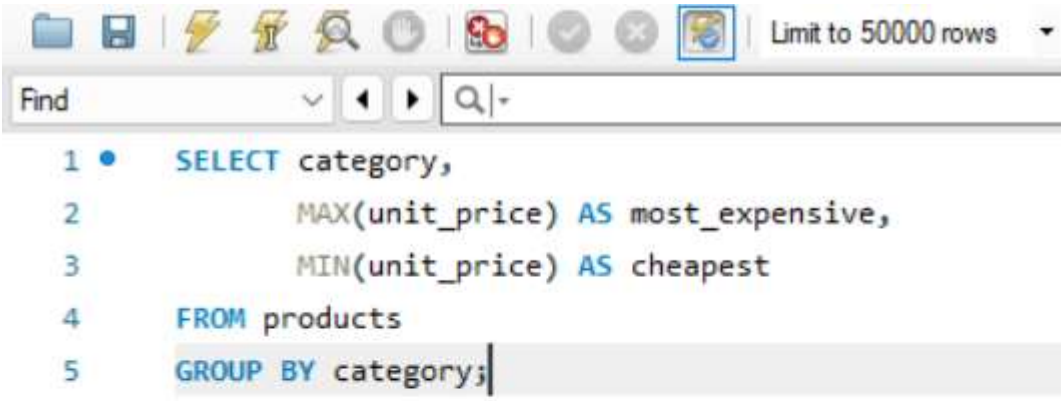
- COUNT(*) counts orders for each status.
- SUM(total_amount) calculates total revenue.
- GROUP BY status groups records according to status.
- ORDER BY total_revenue DESC sorts from highest to lowest revenue.

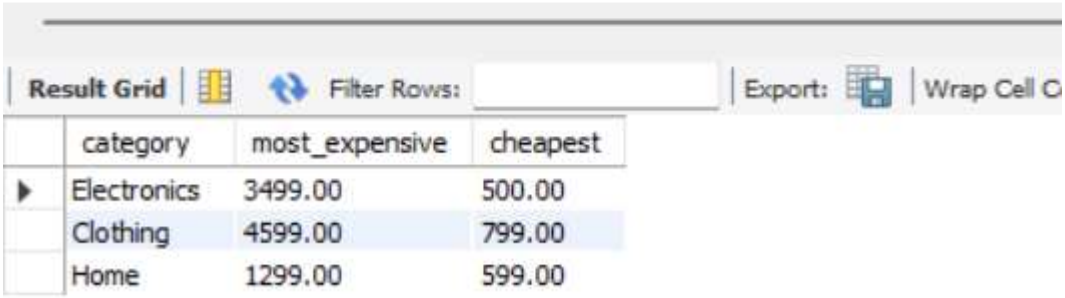
Conclusion

Delivered orders generated the highest revenue.

Q17. Find the most expensive (MAX) and cheapest (MIN) product in each category.

ANS:- SQL QUERY WITH OUTPUT





category	most_expensive	cheapest
Electronics	3499.00	500.00
Clothing	4599.00	799.00
Home	1299.00	599.00

Explanation

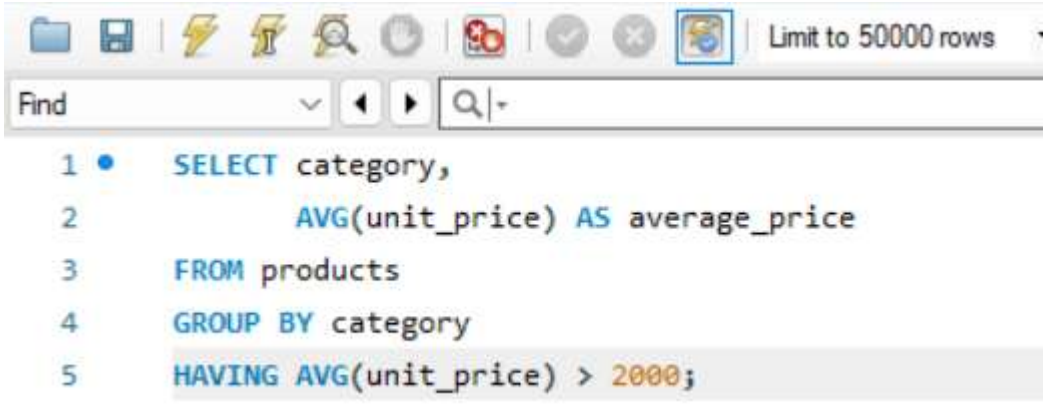
- MAX() returns the highest price in each category.
- MIN() returns the lowest price in each category.
- GROUP BY category performs the calculation separately for each category.

Conclusion

This query helps identify the price range of products within each category.

Q18. List all product categories where the average unit_price is greater than ₹2000.

ANS:- SQL QUERY WITH OUTPUT



Explanation

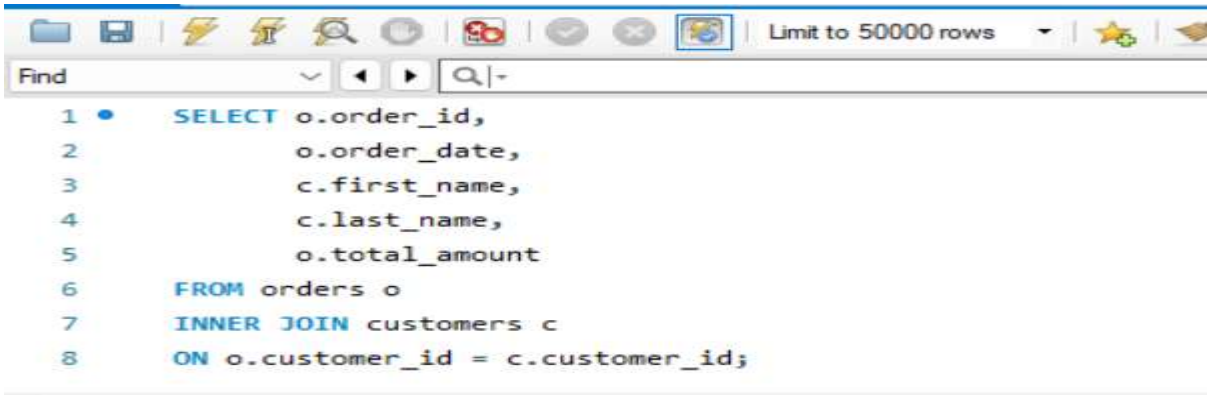
- GROUP BY category groups products by category.
- AVG(unit_price) calculates the average price.
- HAVING filters groups after aggregation.
- Only categories with an average price greater than ₹2000 are displayed.

SECTION – D

JOINS & RELATIONSHIPS

Q19. Write an INNER JOIN query to display each order along with the customer's first_name and last_name. Show: order_id, order_date, first_name, last_name, total_amount.

ANS:- SQL QUERY WITH OUTPUT



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

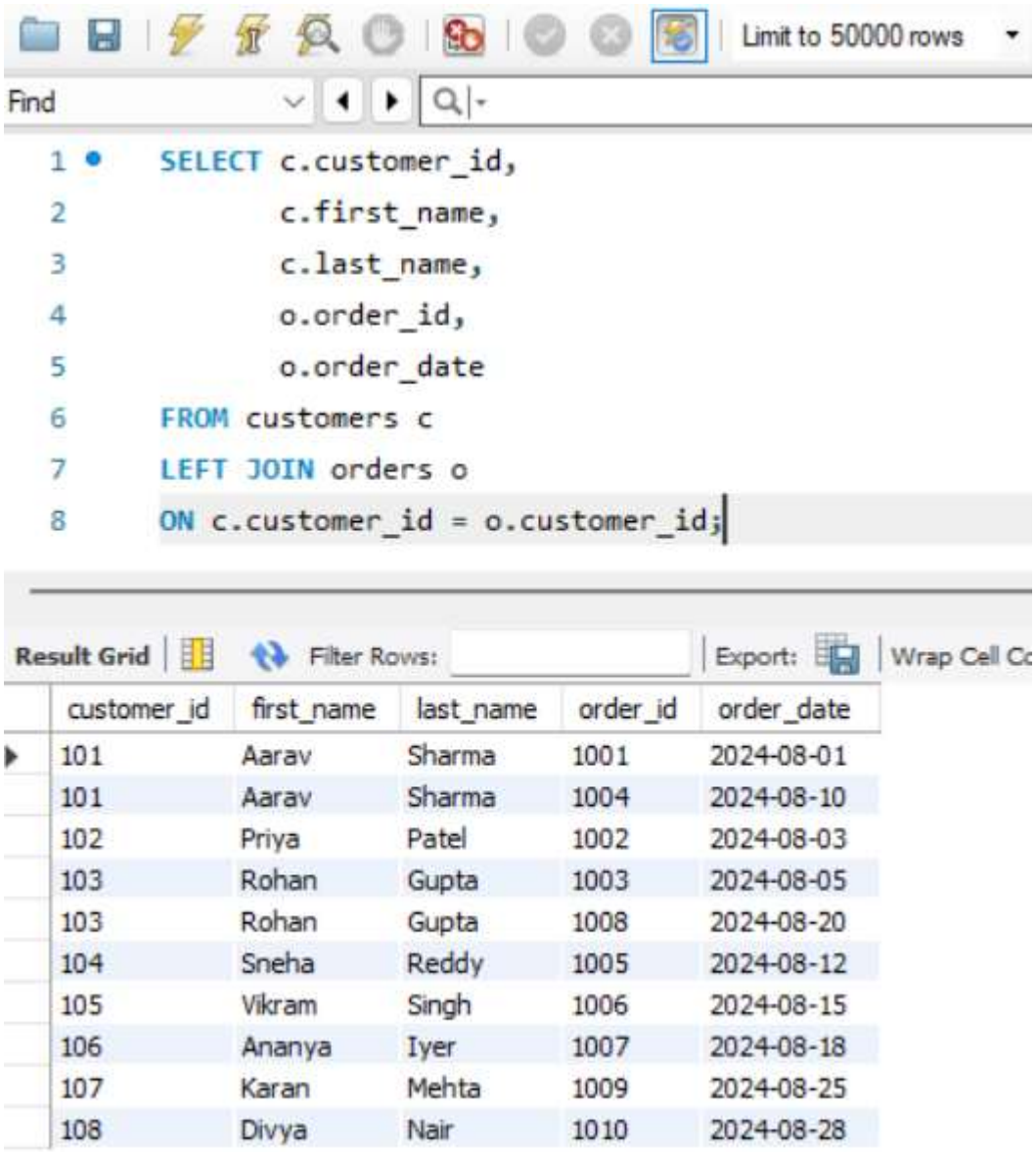
order_id	order_date	first_name	last_name	total_amount
1001	2024-08-01	Aarav	Sharma	4498.00
1004	2024-08-10	Aarav	Sharma	3499.00
1002	2024-08-03	Priya	Patel	799.00
1003	2024-08-05	Rohan	Gupta	7498.00
1008	2024-08-20	Rohan	Gupta	899.00
1005	2024-08-12	Sneha	Reddy	2999.00
1006	2024-08-15	Vikram	Singh	5898.00
1007	2024-08-18	Ananya	Iyer	1299.00
1009	2024-08-25	Karan	Mehta	6098.00
1010	2024-08-28	Divya	Nair	1598.00

Explanation

- INNER JOIN returns only matching records from both tables.
- Orders are joined with customers using customer_id.
- Displays order details along with customer names.

Q20 sing a LEFT JOIN, list ALL customers and their orders (if any). Customers with no orders should still appear with NULL values for order columns.

ANS:- SQL QUERY WITH OUTPUT



customer_id	first_name	last_name	order_id	order_date
101	Aarav	Sharma	1001	2024-08-01
101	Aarav	Sharma	1004	2024-08-10
102	Priya	Patel	1002	2024-08-03
103	Rohan	Gupta	1003	2024-08-05
103	Rohan	Gupta	1008	2024-08-20
104	Sneha	Reddy	1005	2024-08-12
105	Vikram	Singh	1006	2024-08-15
106	Ananya	Iyer	1007	2024-08-18
107	Karan	Mehta	1009	2024-08-25
108	Divya	Nair	1010	2024-08-28

Explanation

- LEFT JOIN returns all records from the left table (customers).
- Matching records from orders are displayed.
- If a customer has no orders, order columns show NULL.

Example of a Customer with No Orders

```sql
INSERT INTO customers
VALUES
(
109,
'Rahul',
'Verma',
'rahul@email.com',
'Delhi',
'Delhi',
'2024-09-01',
FALSE
);
```

OUTPUT

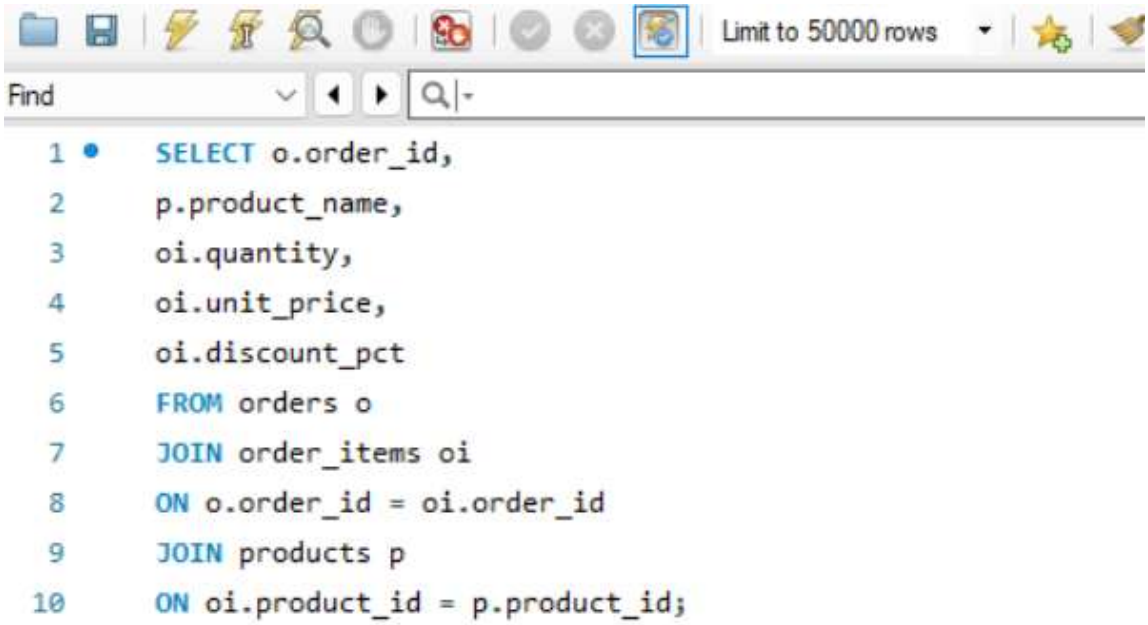
customer_id	first_name	last_name	order_id	order_date	status
109	Rahul	Verma	NULL	NULL	NULL

Conclusion

LEFT JOIN is used when we want **all records from the first table**, even if there are no matching records in the second table. It is useful for identifying customers who have not placed any orders.

Q21. Write a query using JOINs across three tables (orders → order_items → products) to show: order_id, product_name, quantity, unit_price, and discount_pct for each order item.

ANS:- SQL QUERY WITH OUTPUT



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

order_id	product_name	quantity	unit_price	discount_pct
1001	Wireless Earbuds	2	1499.00	0.00
1006	Wireless Earbuds	1	1499.00	10.00
1002	Cotton T-Shirt	1	799.00	0.00
1003	Smart Watch	1	2999.00	0.00
1005	Smart Watch	1	2999.00	0.00
1003	Running Shoes	1	4599.00	5.00
1006	Running Shoes	1	4599.00	5.00
1004	Bluetooth Speaker	1	3499.00	0.00
1009	Bluetooth Speaker	1	3499.00	0.00
1007	Bedsheet Set	1	1299.00	0.00
1010	Bedsheet Set	1	1299.00	0.00
1001	Laptop Stand	1	899.00	10.00
1008	Laptop Stand	1	899.00	0.00
1009	Cushion Covers (...	2	599.00	15.00
1010	Cushion Covers (...	1	599.00	0.00

Explanation

This query joins:

- orders
- order_items
- products

to display complete information about products purchased in each order.

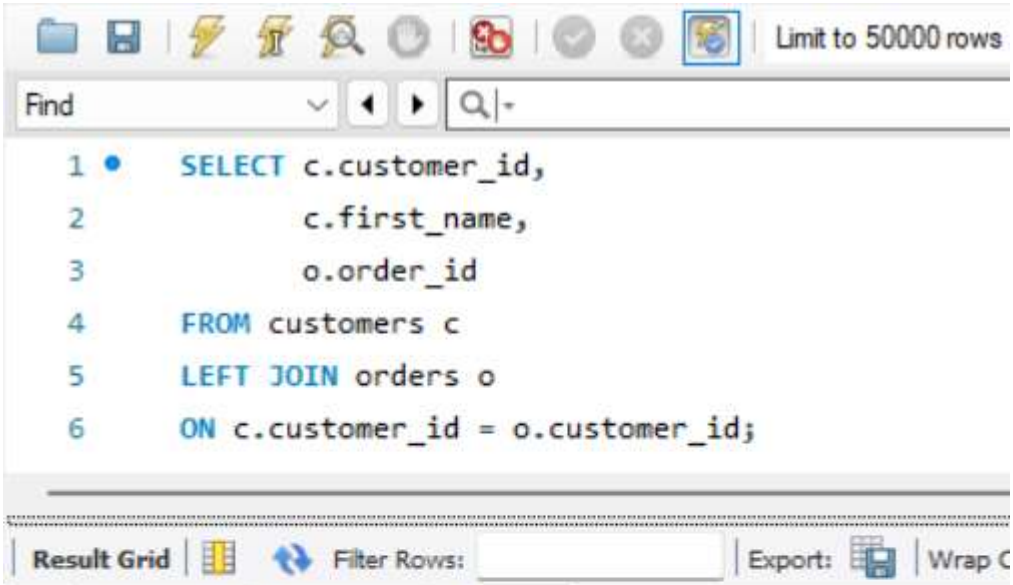
Q22. Explain the difference between LEFT JOIN and RIGHT JOIN. When would you use a FULL OUTER JOIN?

ANS:- **LEFT JOIN**

Returns:

- All records from the left table.
- Matching records from the right table.
- Non-matching records show NULL.

Example



Result Grid | Filter Rows: | Export: | Wrap

customer_id	first_name	order_id
101	Aarav	1001
101	Aarav	1004
102	Priya	1002
103	Rohan	1003
103	Rohan	1008
104	Sneha	1005
105	Vikram	1006
106	Ananya	1007
107	Karan	1009
108	Divya	1010

Result

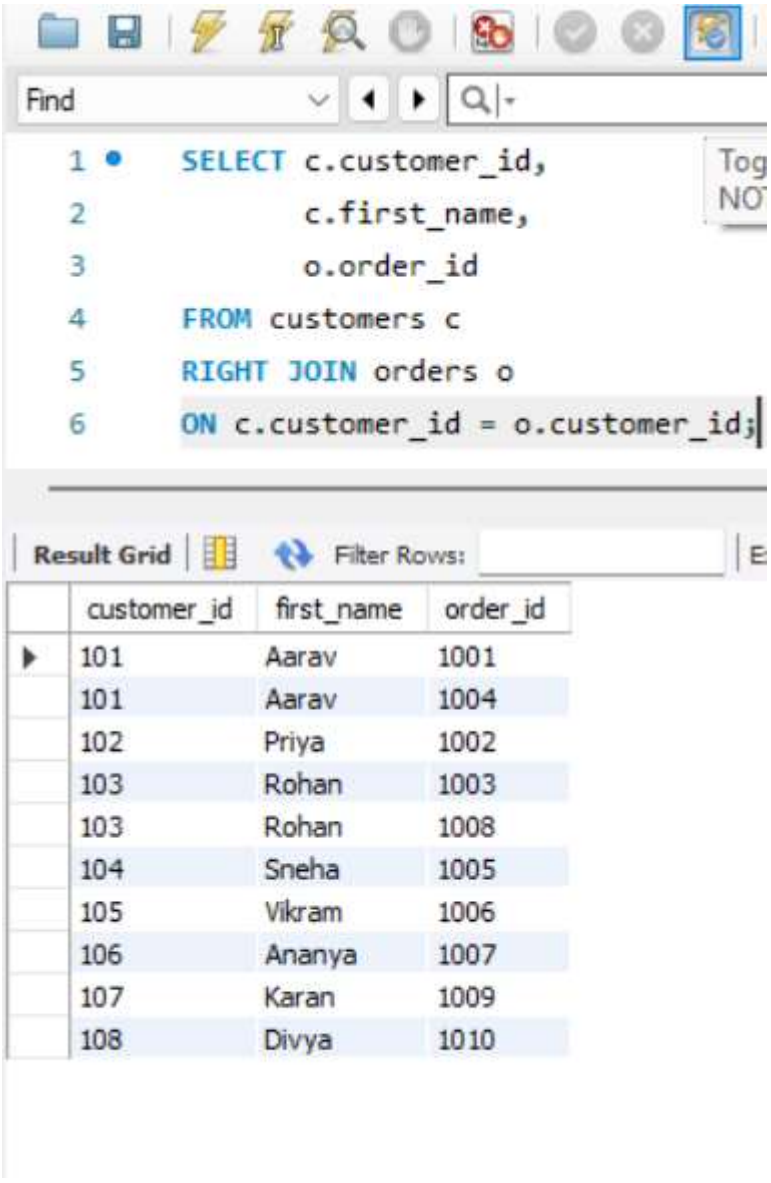
All customers are shown, even if they have no orders.

RIGHT JOIN

Returns:

- All records from the right table.
- Matching records from the left table.
- Non-matching records show NULL.

Example



Result

All orders are shown, even if there is no matching customer.

FULL OUTER JOIN

Returns:

- All records from both tables.
- Matching rows are combined.
- Non-matching rows show NULL values.

When Would You Use It?

Suppose you want:

- All customers
- All orders
- Including customers without orders
- Including orders without customers

Then a FULL OUTER JOIN would be useful.

Note

MySQL does not directly support FULL OUTER JOIN.

Q23. Identify all Foreign Key relationships in the schema. Explain what would happen if you tried to insert an order with customer_id = 999.

ANS:- **Foreign Key Relationships**

Parent Table	Child Table	Foreign Key
customers	orders	customer_id
orders	order_items	Order_id
products	order_items	product_id

Relationship Diagram

customers

|

| customer_id

↓

orders

|

| order_id

↓

order_items

↑

|

Products

Invalid Insert Example

```sql
INSERT INTO orders
VALUES
(
1011,
999,
'2024-09-01',
'Pending',
1000
);
```

OUTPUT

ERROR 1452 (23000):

Cannot add or update a child row:

a foreign key constraint fails

Explanation

- customer_id = 999 does not exist in the customers table.
- The Foreign Key constraint checks whether the customer exists.
- Since customer 999 is not present, MySQL rejects the insertion.

Conclusion

Foreign Keys maintain **referential integrity** by ensuring that related records exist in parent tables before data is inserted into child tables. This prevents invalid and inconsistent data from entering the database.

SECTION – E

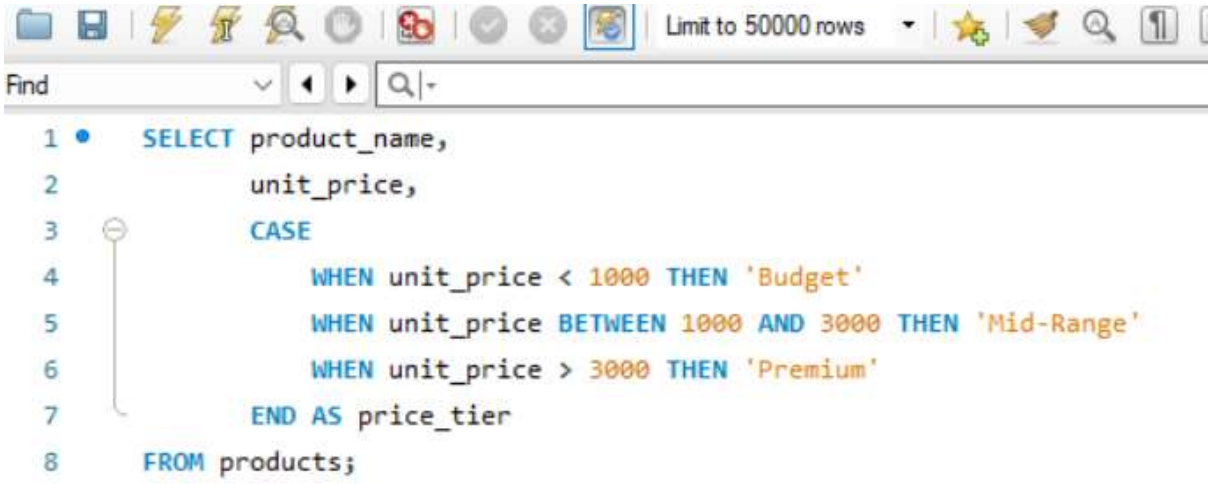
ADVANCED CONCEPTS (CASE, ACID, TRANSACTIONS)

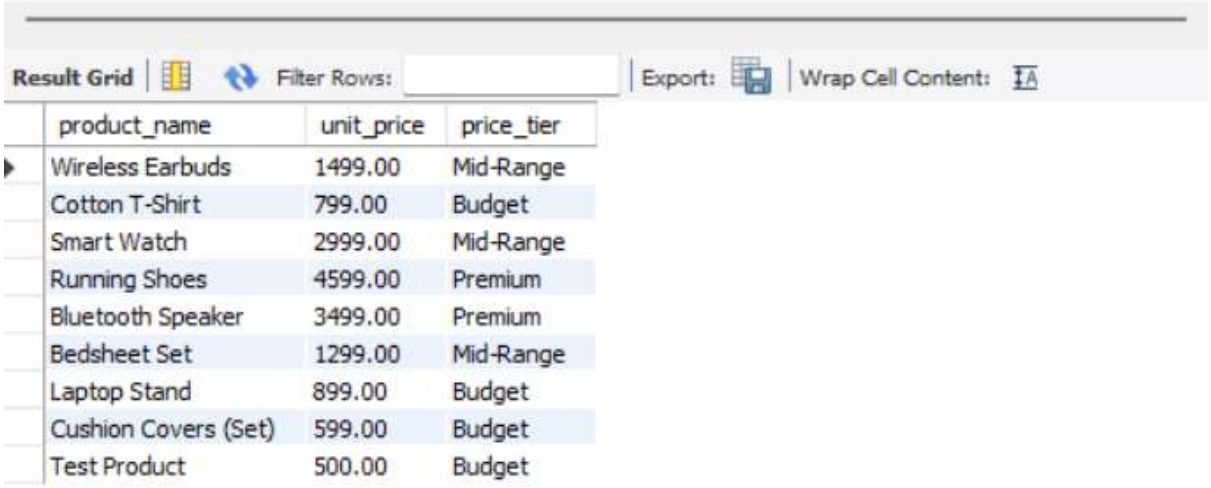
Q24. Write a query using CASE to classify products into price tiers:

- **'Budget' → unit_price < 1000**
- **'Mid-Range' → unit_price BETWEEN 1000 AND 3000**
- **'Premium' → unit_price > 3000**

Display: product_name, unit_price, price_tier

ANS:- SQL QUERY WITH OUTPUT





product_name	unit_price	price_tier
Wireless Earbuds	1499.00	Mid-Range
Cotton T-Shirt	799.00	Budget
Smart Watch	2999.00	Mid-Range
Running Shoes	4599.00	Premium
Bluetooth Speaker	3499.00	Premium
Bedsheet Set	1299.00	Mid-Range
Laptop Stand	899.00	Budget
Cushion Covers (Set)	599.00	Budget
Test Product	500.00	Budget

Explanation

The CASE statement is used to create categories based on product prices.

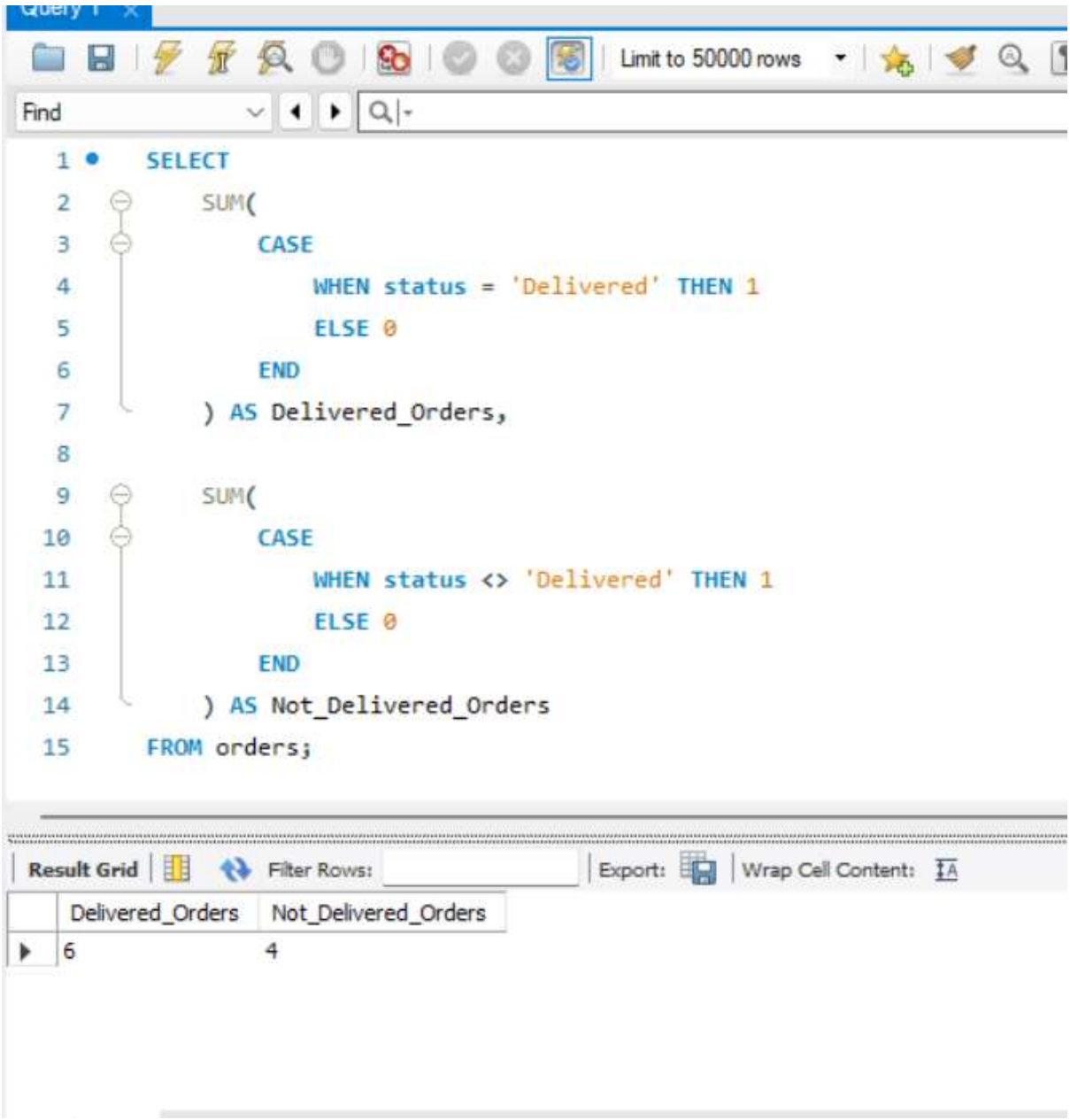
- If unit_price is less than ₹1000, the product is classified as **Budget**.
- If unit_price is between ₹1000 and ₹3000, it is classified as **Mid-Range**.
- If unit_price is greater than ₹3000, it is classified as **Premium**.

Conclusion

This query helps ShopEase categorize products into different price segments, making it easier to analyze product pricing and customer purchasing behavior.

Q25. Using a CASE statement inside an aggregate function, count how many orders are 'Delivered' vs 'Not Delivered' (all other statuses). Display the result in a single row.

ANS:- SQL QUERY WITH OUTPUT



Calculation

Delivered Orders

1001
1002
1004
1006
1008
1010

Total = **6**

Not Delivered Orders

1003 (Shipped)
1005 (Cancelled)
1007 (Pending)
1009 (Shipped)

Total = **4**

Explanation

The CASE statement checks the status of each order:

- If the status is **Delivered**, it returns **1**; otherwise **0**.
- SUM() adds all the 1s to count delivered orders.
- A second CASE statement counts all orders that are not delivered.

This allows both counts to be displayed in a **single row**.

Conclusion

Using CASE inside aggregate functions is a powerful way to create conditional counts and summaries. In the ShopEase database, there are **6 Delivered Orders** and **4 Not Delivered Orders**.

Q26. Explain each letter of ACID:

- **A – Atomicity**
- **C – Consistency**
- **I – Isolation**

- **D – Durability**

Give a real-world example (e.g., bank transfer) showing why each property is important.

ANS:- **ACID** is a set of properties that ensures database transactions are processed reliably and accurately.

ACID stands for:

- **A – Atomicity**
- **C – Consistency**
- **I – Isolation**
- **D – Durability**

A – Atomicity

Meaning

Atomicity means **either all operations of a transaction are completed successfully, or none of them are performed**.

Bank Transfer Example

Suppose ₹1000 is transferred from Account A to Account B.

Steps:

1. Deduct ₹1000 from Account A
2. Add ₹1000 to Account B

If the system crashes after Step 1 but before Step 2, the entire transaction is cancelled.

Why It Is Important

Atomicity prevents partial transactions and ensures that money is not lost during the transfer.

Example

Before Transfer

Account A = ₹5000
Account B = ₹3000

After Transfer

Account A = ₹4000
Account B = ₹4000

If any step fails, both accounts remain unchanged.

C – Consistency

Meaning

Consistency ensures that the database remains in a valid state before and after a transaction.

Bank Transfer Example

Before Transfer:

Account A = ₹5000
Account B = ₹3000

Total Money = ₹8000

After Transfer:

Account A = ₹4000
Account B = ₹4000

Total Money = ₹8000

Why It Is Important

Consistency ensures that business rules and data integrity are maintained.

No money is created or lost during the transaction.

I – Isolation

Meaning

Isolation ensures that multiple transactions running at the same time do not interfere with each other.

Bank Transfer Example

Suppose two customers are transferring money simultaneously.

Transaction 1:

Transfer ₹1000

Transaction 2:

Transfer ₹2000

Each transaction should be processed independently.

Why It Is Important

Isolation prevents incorrect balances and data conflicts when many users access the database at the same time.

D – Durability

Meaning

Durability ensures that once a transaction is committed, it is permanently saved in the database.

Bank Transfer Example

A transfer of ₹1000 is successfully completed and committed.

Immediately after that, the server crashes or power fails.

Why It Is Important

Even after the crash, the transfer information remains stored in the database.

The transaction is not lost.

Example

Transfer Completed

Account A = ₹4000
Account B = ₹4000

After System Crash:

Account A = ₹4000
Account B = ₹4000

The data remains unchanged because the transaction was committed.

Conclusion

ACID properties are essential for maintaining **data accuracy, reliability, and integrity** in database systems. In a bank transfer, Atomicity prevents partial transfers, Consistency maintains correct balances, Isolation avoids conflicts between simultaneous transactions, and Durability ensures completed transactions are permanently stored.

Q27. Write a SQL transaction that does the following atomically:

1. Insert a new order (order_id=1011, customer_id=102, today's date, 'Pending', 1598.00)
2. Insert two order items for that order
3. Update the stock_qty of the purchased products
4. If any step fails, ROLLBACK the entire transaction. Otherwise, COMMIT.
Write the complete BEGIN...COMMIT/ROLLBACK block.

ANS:- A transaction ensures that **all operations are completed successfully together**. If any operation fails, all changes are undone using ROLLBACK.

Insert a new order

Insert two order items

Update product stock

If any step fails, rollback the entire transaction

SQL Transaction

```sql
START TRANSACTION;
-- Step 1: Insert New Order
INSERT INTO orders
( order_id,customer_id,order_date,status,total_amount )
VALUES
(1011,102,CURDATE(),'Pending',1598.00);
-- Step 2: Insert First Order Item
INSERT INTO order_items
(item_id,order_id,product_id,quantity,unit_price,discount_pct)
VALUES
(5016,1011,202,1,799.00,0);
-- Step 3: Insert Second Order Item
INSERT INTO order_items
(item_id,order_id,product_id,quantity,unit_price,discount_pct)
VALUES
(5017,1011,208,1,799.00,0);
-- Step 4: Update Product Stock
UPDATE products
SET stock_qty = stock_qty - 1
WHERE product_id = 202;
UPDATE products
SET stock_qty = stock_qty - 1
WHERE product_id = 208;
-- Step 5: Save All Changes
COMMIT;
```

OUTPUT

Explanation

Step 1: Insert New Order

A new order is created for customer **102**.

```
INSERT INTO orders
VALUES
(
   1011,
   102,
   CURDATE(),
   'Pending',
   1598.00
);
```

Step 2 & 3: Insert Order Items

Two products are added to the order:

- Product 202 (Cotton T-Shirt)
- Product 208 (Cushion Covers Set)

Step 4: Update Stock

The stock quantity of purchased products is reduced by 1.

```
UPDATE products
SET stock_qty = stock_qty - 1
WHERE product_id = 202;
```

```
UPDATE products
SET stock_qty = stock_qty - 1
WHERE product_id = 208;
```

Step 5: Commit Transaction

```
COMMIT;
```

This permanently saves all changes.

If an Error Occurs

ROLLBACK;

All changes made during the transaction are cancelled.

Example

Suppose Product 208 does not exist.

- Order is inserted.
- First order item is inserted.
- Second order item fails.

Without a transaction:

Order remains in database.
Only one item is added.
Database becomes inconsistent.

With a transaction:

ROLLBACK executes.
Order is removed.
Order items are removed.
Stock is unchanged.
Database remains consistent.

Conclusion

Transactions follow the **all-or-nothing principle**. If every step succeeds, COMMIT saves the changes. If any step fails, ROLLBACK restores the database to its previous state, ensuring data accuracy and integrity.